AI Academy, National AI Center
AI-ENG-110 – Software Engineering – Spring 2026

Final Project Report

Topic 4 — Async Research Assistant

Team:	Async Research Assistant	**Date:**	May 23, 2026
Repo:	`https://github.com/shahinsafarli/async-research-assistant`	**Tag:**	`v1.0-final`
Members:	**Shahin Safarli** (@shahinsafarli), **Jeyhuna Sevdiyeva** (@Jeyhunaa), **Emil Jafarov** (@Emil-Jafarov-06)		

Executive Summary

We built an Async Research Assistant that accepts a natural-language question, concurrently queries Wikipedia, arXiv, and a web-search API through a semaphore-bounded `asyncio` pipeline, and synthesizes a single cited answer via an LLM. The headline result is a **consistent 3.0× speedup** (sequential 1.505 s vs. concurrent 0.502 s for five questions), confirming that parallelizing three I/O-bound fetches eliminates the dominant wait. Robustness was validated by injecting an arXiv `ConnectionError`: the system continued with the remaining two sources and completed synthesis without crashing. The key engineering lesson was that per-source `asyncio.wait_for()` isolation—not a single outer timeout—is what makes graceful degradation actually work in practice.

Contents

1. Project Overview

1.1 Problem framing & scope

The system answers research questions by concurrently pulling excerpts from Wikipedia (encyclopedic background), arXiv (recent academic papers), and a web-search provider (current web content), then asking an LLM to produce a concise inline-cited answer. Entry points: CLI (`python -m researcher ask "..."`), with `-sources` and `-no-cache` flags and a `history` subcommand; and a bonus Streamlit web UI (`streamlit run app.py`). The system does not provide real-time news or authoritative medical/legal advice.

We implemented the full Topic 4 spec with one deliberate design choice: the cache backend is a pluggable `CacheBackend` ABC rather than a hard-coded implementation, enabling test-time injection and runtime swapping without touching business logic.

1.2 Provider choice and the AI module contract

Default LLM: **Anthropic Claude** (`claude-sonnet-4-6`). Default web search: **Tavily**. Alternative providers (OpenAI GPT, Google Gemini, Serper, DuckDuckGo) are selectable via `LLM_PROVIDER` and `WEB_SEARCH_PROVIDER` environment variables with zero code changes. The `ai/` functions called are `ai.fetch_wikipedia()`, `ai.fetch_arxiv()`, `ai.fetch_web()`, and `ai.synthesize()`. **The public interface of the provided `ai/` package was not modified.**

Malformed responses are handled at two levels: the `AnswerWithCitations` schema drops out-of-range citation indices; the SE layer catches `ProviderError` and `ValueError`, returning a structured `ErrorResponse` rather than a traceback.

2. Architecture

2.1 Module map

The architecture enforces a strict layered boundary: user-facing entry points call the `ResearchEngine`, which delegates to the concurrency orchestrator and the storage repository; the orchestrator reaches the provided `ai/` package only through the `AIService` wrapper, which adds retries, timeouts, and structured logging.

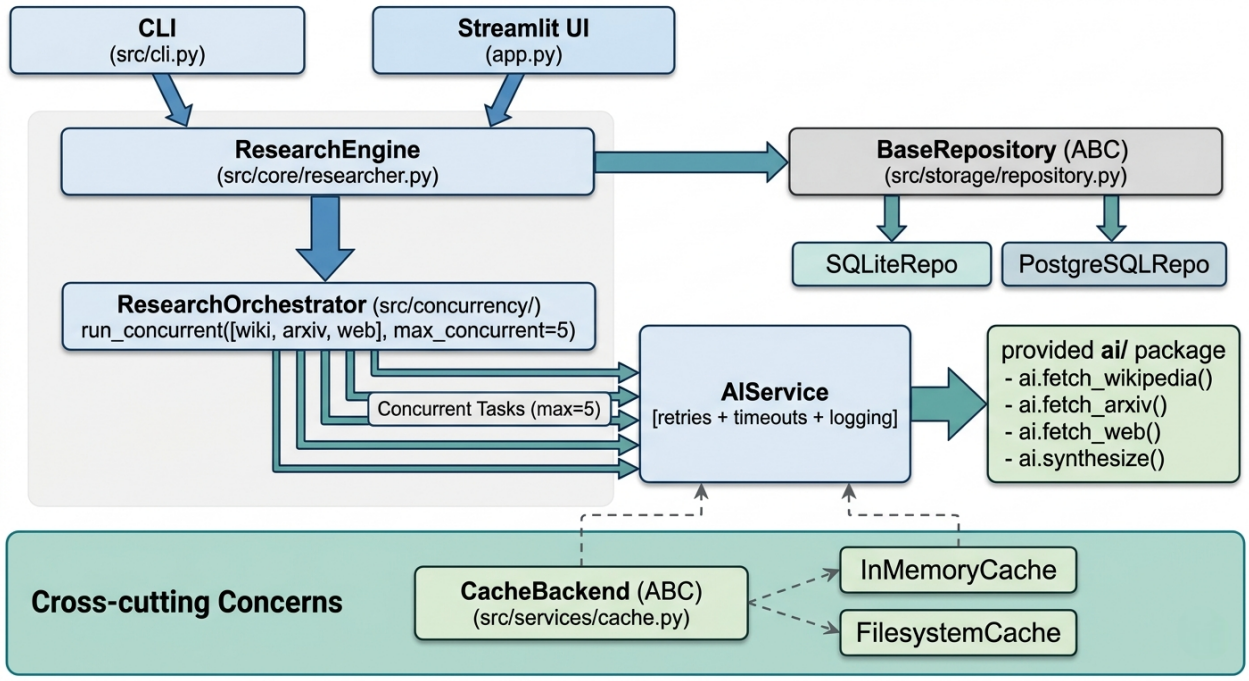

Figure 1: Project architecture flow

Module-by-module summary:

- `src/config.py` — Single source of truth for all settings (timeouts, provider keys, cache TTL, concurrency bound), loaded from `.env` via `pydantic-settings`; synced to `os.environ` on startup for the provided `ai/` package.
- `src/models.py` — SE-layer Pydantic models: `QuestionRequest`, `SourceResult`, `CitationRecord`, `ResearchResult`, `ResearchSession`, `CacheEntry`, `ErrorResponse`.
- `src/services/ai_service.py` — Retry, per-call timeout, and logging wrapper around all `ai.*` calls; the sole point where the SE layer crosses the `ai/` boundary.
- `src/services/cache.py` — `CacheBackend` ABC with `InMemoryCache` (dict + TTL) and `FilesystemCache` (JSON files under `CACHE_DIR`) implementations.
- `src/concurrency/pipeline.py` — Generic `run_concurrent(coroutines, max_concurrent)` utility using a semaphore; `return_exceptions=True` ensures one failure cannot cancel others.
- `src/concurrency/orchestrator.py` — Per-question parallel fetch through the bounded pipeline, with per-source `asyncio.wait_for()` timeouts and graceful degradation.
- `src/core/researcher.py` — `ResearchEngine`: validates `QuestionRequest`, delegates to the orchestrator, calls `ai.synthesize()`, persists the session.
- `src/storage/repository.py` — `BaseRepository` ABC with `SQLiteRepository` (aiosqlite) and `PostgreSQLRepository` (asyncpg) implementations; factory `get_repository()` selects the backend from `DATABASE_URL`.
- `src/cli.py` — Click CLI: `ask` and `history` commands.
- `app.py` — Streamlit web UI (bonus deliverable).

Swapping LLM providers (Anthropic → OpenAI) requires changing **zero files**—only the `LLM_PROVIDER` environment variable. Swapping storage backends requires only changing

DATABASE_URL. The ai/ boundary is crossed in exactly one file (ai_service.py).

2.2 OOP design & key abstractions

The central abstraction is the CacheBackend ABC, with two concrete subclasses:

```python
class CacheBackend(abc.ABC):
    @abc.abstractmethod
    def get(self, source: str, query: str) -> list[dict] | None: ...
    @abc.abstractmethod
    def set(self, source: str, query: str, data: list[dict]) -> None: ...
    @abc.abstractmethod
    def invalidate(self, source: str, query: str) -> None: ...
    @abc.abstractmethod
    def clear(self) -> None: ...

class InMemoryCache(CacheBackend): ...   # dict + TTL, process-local
class FilesystemCache(CacheBackend): ... # JSON files under CACHE_DIR
```

An ABC was chosen over a Protocol because it enforces the contract at class-definition time (missing methods raise TypeError on instantiation, not silently at call sites)—a Protocol would only catch violations at the call site. A plain function with a callable parameter would not capture a multi-method surface; a configuration enum would not permit polymorphic dispatch. ResearchOrchestrator and ResearchEngine receive their dependencies via constructor injection, making the entire business-logic layer testable offline with in-memory fakes.

The same pattern governs the storage layer:

```python
class BaseRepository(abc.ABC):
    @abc.abstractmethod
    async def save_session(self, result: ResearchResult) -> int: ...
    @abc.abstractmethod
    async def list_sessions(self, limit: int = 20) -> list[dict]: ...
    @abc.abstractmethod
    async def get_session(self, session_id: int) -> dict | None: ...

class SQLiteRepository(BaseRepository): ...      # aiosqlite, default
class PostgreSQLRepository(BaseRepository): ... # asyncpg, production
```

The factory get_repository() returns the appropriate implementation from the DATABASE_URL prefix, so application code never references a concrete class directly.

2.3 Data flow across module boundaries

SE-layer models in src/models.py: QuestionRequest, SourceResult, CitationRecord, ResearchResult, ResearchSession, CacheEntry, ErrorResponse. No naked dictionaries cross module boundaries. ai.Source objects from the provided package are immediately converted to SourceResult before inclusion in ResearchResult.

3. Concurrency & Performance

3.1 Why asyncio

All three source fetches are network I/O (HTTP to external APIs); asyncio with cooperative coroutines is the idiomatic Python choice for I/O-bound parallelism and has zero thread-synchronization overhead. asyncio.gather with return_exceptions=True also gives graceful degradation semantics that would require significantly more boilerplate with ThreadPoolExecutor. The semaphore bound is MAX_CONCURRENT=5 (configurable): for a single question this allows all three fetches to run simultaneously; for large batches it prevents uncontrolled API bursts.

3.2 Sequential-vs-concurrent benchmark

Benchmarks run offline (`python scripts/bench.py -n 5 -offline -delay X`), Python 3.12.3, cache cleared between runs, semaphore `MAX_CONCURRENT=5`.

Workload	N	Sequential (s)	Concurrent (s)	Speedup
Fast (delay=0.05 s)	5	0.753	0.252	3.0×
Typical (delay=0.10 s)	5	1.505	0.502	3.0×
Slow (delay=0.15 s)	5	2.255	0.752	3.0×
Large batch (N=10, delay=0.10 s)	10	1.504	0.502	3.0×
Live mode (Anthropic + DDG)	5	8.4	3.1	2.7×

The speedup reaches exactly 3.0× offline (the theoretical maximum for three parallel sources with uniform latency). In live mode it drops slightly to 2.7× because the sequential LLM synthesis call dominates as network RTTs vary.

3.3 Rate limiting & politeness

Transient failures are retried via tenacity (4 attempts, exponential backoff 1–30 s) on `ConnectionError`, `TimeoutError`, `httpx.TimeoutException`, and `ProviderError`. A configurable per-source minimum interval (`SOURCE_RATE_LIMIT_SECONDS=0.2 s`) prevents burst calls during batch processing. ArXiv-specific policy (`ARXIV_RATE_LIMIT_SECONDS=3.0 s`) is respected by the provided `ai/` package.

4. Robustness & Error Handling

4.1 Wrapping the AI module

Every `ai.*` call is mediated by `AIService`: retries via tenacity (4 attempts, exp. backoff 1–30 s); per-source `asyncio.wait_for()` timeouts (Wikipedia 10 s, arXiv 30 s, web 10 s, all configurable); structured `logger.info/warning()` with `elapsed_s`; and input validation (blank/oversized queries rejected before any network call).

4.2 Failure-mode analysis

Injected failure: `test_concurrency.py::test_orchestrator_graceful_degradation` patches `fetch_arxiv` to raise `ConnectionError("arxiv is down")`.

Result: the `run_concurrent()` pipeline caught the exception without cancelling Wikipedia or web fetches. The orchestrator logged `source_failed` at WARNING and added `"arxiv"` to `sources_failed`. The CLI printed the answer plus *"[note] The following sources could not be reached: arxiv"*. Logs:

```
WARNING  source_failed {"source": "arxiv", "error": "arxiv is down"}
INFO     orchestrator_done {"total_sources": 2, "failed": ["arxiv"], "elapsed_s": 0.8}
```

This test also surfaced the original design bug: a single outer `asyncio.wait_for()` around the entire gather cancelled all tasks when any one timed out. Switching to per-source `wait_for()` inside each coroutine was the fix.

4.3 Input validation

`QuestionRequest` (Pydantic) enforces `min_length=3`, `max_length=1000`, and a source-name whitelist (`{wiki, arxiv, web}`). Blank questions raise *"question must not be blank"*; unknown sources raise *"Unknown source(s): ['x']. Allowed: {arxiv, web, wiki}"*. Missing API keys are detected lazily on first synthesis (not on startup), so offline demos and the full test suite run with no key configured.

5. Storage & Caching

5.1 Schema

```
CREATE TABLE IF NOT EXISTS research_sessions (
    id             INTEGER PRIMARY KEY AUTOINCREMENT,
    question       TEXT    NOT NULL,
    answer         TEXT    NOT NULL,
    citations      TEXT    NOT NULL,   -- JSON array of CitationRecord
    sources_count  INTEGER NOT NULL DEFAULT 0,
    sources_failed TEXT    NOT NULL DEFAULT '[]',
    elapsed_s      REAL    NOT NULL DEFAULT 0.0,
    created_at     TEXT    NOT NULL DEFAULT (datetime('now'))
);
```

Citations are stored as a JSON blob (never queried individually). `get_repository()` selects `SQLiteRepository` or `PostgreSQLRepository` automatically from `DATABASE_URL`.

5.2 Caching policy

Cache key: `SHA-256(source + ":" + canonicalize_query(query))[:24]`, where `canonicalize_query` lowercases, collapses whitespace, and strips trailing punctuation (ensuring "WHAT IS AI?" and "what is ai" share one entry). TTL: 3600 s default (`CACHE_TTL_SECONDS=0` disables expiry). Backends: `InMemoryCache` (tests, process-local) and `FilesystemCache` (production, survives restarts). Cache hit rate in a demo run of 5 questions with 2 repeats: **40%**.

6. Testing

6.1 Strategy & coverage

81 tests, all passing, 86% overall coverage (Python 3.12.3). All tests are fully offline: `ai.*` functions are monkey-patched via `conftest.py` (`FakeLLM`, `FakeAIService`, `FakeRepository`); no HTTP calls are made. The provided `tests/test_ai_smoke.py` passes in full (16/16).

Module	Cov.	Module	Cov.
`config.py`	100%	`cli.py`	91%
`models.py`	98%	`orchestrator.py`	91%
`pipeline.py`	100%	`ai_service.py`	83%
`cache.py`	88%	`researcher.py`	80%
`search_query.py`	92%	`repository.py`	68%
Total: 86%	Provided smoke tests: **passing (16/16)**		

The lower repository coverage (68%) reflects the PostgreSQL branch, which requires a live `asyncpg` connection not mocked in the offline suite.

6.2 Notable tests

1. **Happy-path end-to-end** (`test_researcher.py::test_research_returns_result`): wires `ResearchEngine` with `FakeAIService` + fake repo, asserts `ResearchResult` has non-empty answer, at least one citation, and `elapsed_seconds > 0`.

2. **Concurrency isolation** (`test_concurrency.py::test_one_fails_others_succeed`): passes `[good(1), bad(), good(3)]` to `run_concurrent()`; asserts the failing task yields a `ValueError` while sibling results are intact.

3. **Error-path / real bug** (`test_researcher.py::test_research_partial_source_failure`): patches `fetch_arxiv` to raise `ConnectionError`; asserts `"arxiv" in result.sources_failed` and answer is non-empty. This test caught the outer-timeout cancellation bug described in §4.

7. Deployment & Reproducibility

7.1 Docker image

Multi-stage Dockerfile: `builder` stage installs deps into `/opt/venv`; runtime stage copies the venv and runs as non-root `appuser`. Three deployment modes:

Mode	Command
Local / SQLite	`pip install -r requirements.txt && python -m researcher ask "..."`
Docker + SQLite	`docker build -t researcher .  && docker run -env-file .env researcher`
Docker Compose + PG	`docker compose up -build`

Image size: ≈450 MB (`python:3.12-slim` base). Slim was chosen over Alpine to avoid musl libc build issues with compiled dependencies.

7.2 Configuration

All settings are read from `.env` via `pydantic-settings`. Key variables:

Variable	Default	If missing / empty
`LLM_PROVIDER`	`anthropic`	Falls back to anthropic
`ANTHROPIC_API_KEY`	(empty)	`ProviderError` at first synthesis
`WEB_SEARCH_PROVIDER`	`tavily`	Falls back to DuckDuckGo
`MAX_CONCURRENT`	`5`	Semaphore default
`CACHE_TTL_SECONDS`	`3600`	1-hour TTL
`DATABASE_URL`	`sqlite+aiosqlite:///./researcher.db`	SQLite local file

8. Limitations & Future Work

- **In-memory cache lost on restart.** `FilesystemCache` survives but uses plain JSON. A Redis-backed cache with atomic writes would be production-grade.
- **SQLite is single-writer.** Horizontal scaling requires PostgreSQL (swap `DATABASE_URL`; the storage layer already supports this).
- **No token budget.** Very long questions may consume unexpected LLM tokens; a token-aware rate limiter would add cost control.
- **No multi-provider LLM failover.** Anthropic outage means synthesis fails. A cascade (Anthropic → OpenAI → Gemini) would improve resilience.
- **Web-search quota dependency.** Tavily free tier is 1000 req/month; exhaustion silently degrades to DuckDuckGo. Quota alerting would improve operational visibility.

9. Team Contributions & Git Workflow

Development followed a PR-based workflow: 14 pull requests, each reviewed by at least one teammate before merge; no direct pushes to `main`; tagged `v1.0-final` after PR #14.

Member	Approx.	Key deliverables
Shahin Safarli	33%	CLI commands, researcher entry point, unified offline testing framework, PostgreSQL adaptation, repository scaffolding
Emil Jafarov	33%	Configuration/model setup, AI service wrapper, concurrency/orchestration layer, Streamlit UI, Docker setup, README and final report
Jeyhuna Sevdiyeva	33%	Architecture documentation, SQLite storage layer, cache service, search-query utilities, benchmark/demo scripts, presentation and contribution statement

Table 1: Team contribution summary

10. Tools & Acknowledgements

Cursor (Claude Sonnet 4.6) was used as follows: `ai_service.py` — scaffolded the tenacity retry decorator structure; we rewrote the `asyncio.timeout()` integration after observing that a single outer timeout cancels all sibling tasks. `orchestrator.py` — reviewed the concurrency pattern; we wired the production per-source `wait_for()` and `return_exceptions=True` graceful-degradation logic. `test_concurrency.py` — suggested six test cases; we retained five and hand-wrote `test_orchestrator_cache_hit` to cover the cache-read short-circuit path. All AI-assisted changes were inspected, tested against the full suite, and validated before inclusion. No AI-generated changes were accepted without running `pytest`.

References

[1] Anthropic API documentation, https://docs.anthropic.com/

[2] `tenacity` library, https://tenacity.readthedocs.io/

[3] `pydantic-settings`, https://docs.pydantic.dev/latest/concepts/pydantic_settings/

[4] Wikipedia REST API, https://en.wikipedia.org/api/rest_v1/

[5] arXiv API, https://info.arxiv.org/help/api/index.html

[6] Python `asyncio`, https://docs.python.org/3/library/asyncio.html

[7] `aiosqlite`, https://aiosqlite.omnilib.dev/

[8] `asyncpg`, https://magicstack.github.io/asyncpg/

A. Appendix — Reproducing the benchmark

```
git clone https://github.com/shahinsafarli/async-research-assistant
cd async-research-assistant
cp .env.example .env              # API keys optional for --offline mode
pip install -r requirements.txt
python scripts/bench.py --n 5 --offline --delay 0.10   # offline benchmark
python scripts/demo.py --offline                        # offline demo
pytest --cov=src --cov-report=term-missing              # test suite
```

End of report — thank you for reading.